Segment Platform Research Foundation Document

Browser-Based Interactive 3D Medical Segmentation with SAM-Med3D-turbo

Steffen Lukas steffen.lukas@charite.de

18 March 2026

Contents

Abstract

This document is a technical and research foundation for the Segment platform, a browser-based 3D medical segmentation environment built around SAM-Med3D-turbo. It defines the clinical targets, model assumptions, workflow patterns, system architecture, and evaluation path for two initial application domains: coronary CCTA in DISCHARGE and prostate mpMRI. It is a research planning document and technical reference rather than a deployment certification package.

Purpose

This reference serves as the working document for framing the segmentation problem, the platform requirements, and the validation roadmap for Charité-led development.

It includes architecture and deployment notes only where they directly affect segmentation planning, validation, and implementation sequencing.

It is **not**:

- a standalone production deployment architecture specification;
- a finished browser-platform product brief;
- a standalone funding proposal.

Research use only — not for clinical decision-making. All segmentation code, HU thresholds, stenosis formulae, and AI model configurations in this document are research prototypes. They have not been validated for clinical use, are not CE-marked or FDA-cleared, and must not be used to inform clinical decisions about individual patients without independent expert review and regulatory approval.

Why Segmentation Matters Here

The current programme needs segmentation for three distinct but linked tasks:

1. **Coronary lumen geometry** for stenosis quantification, centreline extraction, and straightened MPR analysis.
2. **Outer-wall and plaque context** where vessel-wall estimation and plaque-burden measurements are clinically relevant.
3. **Prostate zones, lesions, and organs at risk** for biopsy support, radiotherapy planning, and multi-domain testing of foundation-model generalisation.

Document Snapshot

Field	Detail
Institution	Charité - Universitätsmedizin Berlin
Core Stack	TypeScript · Vite · Niivue 0.66 · FastAPI · PyTorch
Foundation Model	SAM-Med3D-turbo (3D ViT-B/16, 91 M params)
Primary Dataset	DISCHARGE (~25 M DICOM slices across 3,561 patients)
External Validation Dataset	SCOT-HEART (4,146 patients)
Pre-training Corpus	SA-Med3D-140K (21 729 volumes, 143 518 masks, 245 categories)
Clinical Domains	Coronary CCTA (lumen / outer wall / plaque) · Prostate mpMRI (zones / lesions / OARs)

0.1 Executive Summary & Research Vision

0.1.1 Clinical Gap

Problem	Impact
Manual coronary segmentation	30–60 min / case, €200–400
Limited scalability	DISCHARGE (3,561 patients) still largely un-segmented; SCOT-HEART external validation remains to be established

Problem	Impact
Prostate mpMRI zone/lesion contouring	Equally time-intensive for biopsy/radiotherapy planning
Centre-specific expertise	Manual segmentation available only at specialised sites

0.1.2 Our Solution

A **single, universal, browser-based platform** powered by **SAM-Med3D-turbo**:

- **1–3 point prompts** (3D coordinates in mm space) → any anatomy, any modality
- **Zero installation** — runs in Chrome / Firefox / Safari
- **On-premise GPU inference** at Charité (GDPR-compliant; no data leaves the hospital)
- **Interactive workflow** for radiologists + **batch research mode** for DISCHARGE processing + **active-learning loop** for continuous improvement

0.1.3 Research Foundation

This platform is a **test-bed for foundation-model research** in cardiovascular imaging:

1. Evaluate **zero-shot / few-shot generalisation** of SAM-Med3D-turbo on unseen DISCHARGE cases, then test external generalisation on SCOT-HEART
2. Quantify **active-learning gains** (weekly fine-tuning on expert corrections)
3. Benchmark against **nnU-Net v2**[1] task-specific models on segmentation-derived endpoints (stenosis %, plaque burden, myocardium volume)
4. Open-source modular components for the broader MedAI community

0.1.4 Success Targets (6-month horizon)

Metric	Target	Notes
DISCHARGE cases auto-processed	> 80 % of dataset	Batch mode
End-to-end latency	< 2 s per click (interactive)	Single-patch response; full vessel via background queue (10–60 s)
DSC (coronary lumen)	> 0.85 vs. expert	Phase 4 fine-tuned target
DSC (prostate whole-gland)	> 0.90 vs. expert	Phase 4 fine-tuned target
Cost reduction	10× cheaper than manual contouring	Based on 30–60 min manual vs. < 5 min AI-assisted

0.1.5 Evaluation Metrics & Go/No-Go Thresholds

Segmentation quality is reported as **DSC (Dice Similarity Coefficient)** and **HD95 (95th-percentile Hausdorff Distance, mm)**. “Dice” and “DSC” are used interchangeably throughout this document;

all numeric targets refer to DSC. HD95 is essential for coronary work: a high-Dice mask with a single missed distal branch creates a large HD95 spike that would fail clinical acceptance.

Structure	Phase 2 gate (zero-shot)	Phase 4 gate (fine-tuned)	HD95 max (mm)	Notes
LV myocardium (CCTA)	DSC ≥ 0.82	DSC ≥ 0.88	≤ 3.0	Closest to ACDC pre-training — use as first zero-shot benchmark
Coronary lumen (LAD/RCA/LCx)	DSC ≥ 0.70	DSC ≥ 0.85	≤ 2.0	Primary clinical target
Outer wall / EEM	DSC ≥ 0.65	DSC ≥ 0.80	≤ 2.5	Requires dense-prompt implementation
Diameter stenosis %	r ≥ 0.80 vs. QAngio CT	r ≥ 0.90 vs. QAngio CT	—	Pearson correlation; bias < 5 %
Prostate whole gland (T2W)	DSC ≥ 0.85	DSC ≥ 0.92	≤ 3.0	Large structure; high zero-shot prior expected
Prostate peripheral zone	— (fine-tuning first)	DSC ≥ 0.80	≤ 4.0	Subtle PZ–TZ boundary

Structure	Phase 2 gate (zero-shot)	Phase 4 gate (fine-tuned)	HD95 max (mm)	Notes
Prostate suspicious lesion	—	DSC ≥ 0.60	≤ 5.0	Small target; predicted vs. reference lesion volume (mL) Pearson correlation as secondary clinical metric

Phase gate rule: a phase transition is blocked if *any* mandatory structure falls below the Phase N gate threshold on the held-out DISCHARGE validation split (n = 100 cases, fixed before Phase 3 begins, not used for fine-tuning).

> **Critical note on Dice targets:** The SAM-Med3D paper reports **87.12 % Dice on cardiac structures** with 1 prompt point (Table 5 in[2]). However, this was measured on the ACDC dataset (cardiac MRI short-axis cine), **not** coronary CTA. Coronary arteries are smaller, noisier, and motion-affected — published coronary lumen Dice values for task-specific models (nnU-Net[3]) range 0.75–0.88 depending on vessel branch. Our 0.85 target is therefore ambitious but grounded.
>
> **Hardware & deployment requirements** (GPU specs, production cluster, GDPR constraints) are in Deployment, Performance & Security.

0.2 Foundation Model: SAM-Med3D-turbo — Verified Technical Profile

0.2.1 Paper & Publication

Field	Value
Title	SAM-Med3D: Towards General-purpose Segmentation Models for Volumetric Medical Images

Field	Value
Authors	Haoyu Wang, Sizheng Guo, Jin Ye, Zhongying Deng, Junlong Cheng, Tianbin Li, Jianpin Chen, Yanzhou Su, Ziyan Huang, Yiqing Shen, Bin Fu, Shaoting Zhang, Junjun He, Yu Qiao
Venue	**ECCV BIC 2024 — Oral**
arXiv	2310.15161
License	Apache 2.0

0.2.2 Architecture (Verified from Paper §4.1 & GitHub)

SAM-Med3D uses a **fully 3D architecture trained from scratch** (Method 3 in the paper). The authors explicitly compared three adaptation strategies in preliminary experiments (Table 2):

Strategy	Seen Dice	Unseen Dice	Chosen?
3D Adapter + Frozen SAM	Lower	Moderate	No
Fine-tune SAM 2D→3D weights	Good	Poor (broken priors)	No
Train 3D from scratch	**Good**	**Best**	Yes

Rationale (Paper §4.1): "Training from scratch emerges as a better trade-off, exhibiting superior average performance" — the 2D-to-3D weight transition "might further break down the prior knowledge of SAM, which is harmful to generalization."

Component breakdown:

Component	Architecture	Parameters
Image Encoder	3D ViT-B/16 (3D positional encoding, 3D convolutions, 3D LayerNorm)	~86 M
Prompt Encoder	3D point/box encoder (learned embeddings)	~1 M
Mask Decoder	Lightweight 3D decoder (2-layer transformer + upsampling)	~4 M
Total		**~91 M**

> **Critical note:** The paper states "86M encoder + 5M decoder" in various summaries. The exact split varies by source. The model is **not** a modified SAM (Meta) — it is architecturally distinct, sharing only the conceptual prompt-based paradigm.

0.2.3 Training (Verified from Paper §4.2)

Two-stage procedure:

Stage	Data	Epochs	Purpose
Stage 1: Pre-training	All 131 K masks from SA-Med3D-140K training set	800	Build general 3D medical understanding
Stage 2: Fine-tuning	~75 K filtered high-quality masks	Additional	Improve prompt efficiency on challenging targets

SAM-Med3D-turbo (from GitHub issue #2): - Fine-tuned on **44 additional datasets** beyond the base SA-Med3D-140K - Optimised for **sub-second inference** with FP16 - This is the recommended checkpoint for deployment

0.2.4 SA-Med3D-140K Dataset (Verified from HuggingFace)

Statistic	Value
Total 3D images	21 729
Total 3D masks	143 518
Anatomical categories	245
Modalities	28 (CT, MR, US, and more)
Sources	70 public datasets + 8,128 privately licensed cases from 24 hospitals
Primary task	General-purpose promptable segmentation

0.2.5 Verified Performance (from Paper Tables 3–5)

Overall (Table 3): - SAM-Med3D with 1 point: **+60.12 % overall Dice** improvement over original SAM - Consistently outperforms SAM-Med2D across all prompt counts - Operates at **1–26 % of inference time** compared to slice-by-slice SAM

By anatomy (Table 5, 1 prompt point):

Anatomy	SAM	SAM-Med2D	SAM-Med3D
Cardiac (seen)	Poor	Moderate	**87.12 %**
Organs (seen)	Poor	Moderate	**Best**
Lesions (unseen)	Very poor	Moderate	Competitive
Bones/muscles	Very poor	Moderate	**Greatest advantage**

Key finding (Paper §5.1.5): “SAM-Med3D using 1 point outperforms SAM-Med2D with N points in **45 targets out of 49**, achieving up to +68.2% improvement.”

Transferability (Paper §5.1.6, Table 6): When SAM-Med3D's ViT encoder is used as pre-trained backbone for UNETR, it yields up to **+5.63 % Dice improvement** over training from scratch — confirming value as a foundation model.

> **Critical caveat for our project:** The "cardiac" results (87.12 %) are from the **ACDC dataset** (cardiac MRI, short-axis cine — segmenting LV/RV/myocardium). This is **not** coronary artery segmentation from CTA. Coronary arteries are 1.5–4 mm diameter, motion-affected, and require dual-wall segmentation — a substantially harder task not directly evaluated in the paper. The model has **never been benchmarked on coronary CTA lumen segmentation**. Our project will provide this missing evaluation.
>
> **Strategic implication:** Because LV/RV/myocardium from cardiac MRI *is* the dominant pre-trained cardiac target, **myocardial segmentation from CCTA should be the first zero-shot validation task** — not coronary lumen. It is the easiest coronary-domain target for the model given pre-training priors, provides a large structure for establishing an initial Dice baseline, and gives a lower-risk first test of CCTA cardiac generalisation. The coronary lumen evaluation should follow once the myocardial baseline is established.

0.2.6 Official Resources

Resource	Link
GitHub	uni-medical/SAM-Med3D
Paper	arXiv:2310.15161
Supplementary	ECCV Supplementary PDF
Turbo checkpoint	HuggingFace: sam_med3d_turbo.pth
Dataset	HuggingFace: SA-Med3D-140K
MedIM loader	uni-medical/MedIM
CVPR25 Challenge	MedSegFM Competition

0.2.7 Environment Setup (Verified from GitHub README)

Model / research environment:

```
conda create --name sammed3d python=3.10
conda activate sammed3d
pip install uv
uv pip install torch==2.6.0 torchvision==0.21.0 torchaudio==2.6.0
uv pip install torchio opencv-python-headless matplotlib prefetch_generator monai e
```

Backend (FastAPI server) — additional dependencies:

```
uv pip install fastapi uvicorn nibabel SimpleITK celery redis
```

Compile this document to PDF:

```
./code/md2pdf.sh plan/sam3d.md
# Optional: also produce DOCX
./code/md2pdf.sh plan/sam3d.md --docx
```

0.2.8 Model Loading (Verified from GitHub)

```
import medim

# Option A: Direct from HuggingFace (downloads automatically)
# Use /resolve/ (raw file), NOT /blob/ (HTML page)
ckpt_path = "https://huggingface.co/blueyo0/SAM-Med3D/resolve/main/sam_med3d_turbo.
model = medim.create_model("SAM-Med3D", pretrained=True, checkpoint_path=ckpt_path)

# Option B: Local checkpoint (recommended for deployment)
model = medim.create_model(
    "SAM-Med3D",
    pretrained=True,
    checkpoint_path="app/models/checkpoints/sam_med3d_turbo.pth"
)

# Optimise for inference
import torch
device = "cuda" if torch.cuda.is_available() else "cpu"
model = model.to(device)
if device == "cuda":
    model = model.half()  # FP16 for speed + memory on GPU
model.eval()
```

0.2.9 Data Format for Fine-tuning (Verified from GitHub)

SAM-Med3D expects **nnU-Net-style**[3] directory layout with **per-structure binary masks**. Unlike nnU-Net which accepts a single multi-class integer mask (0=background, 1=class1, …), SAM-Med3D requires one separate binary NIfTI file (values 0/1) per anatomical structure. Using a single multi-class mask will silently produce incorrect training prompts.

```
data/medical_preprocessed/
├── coronary/
│   ├── ct_DISCHARGE/
│   │   ├── imagesTr/
│   │   │   ├── discharge_0001.nii.gz
│   │   │   └── ...
│   │   └── labelsTr/
│   │       ├── discharge_0001.nii.gz  (binary mask)
│   │       └── ...
└── prostate/
```

```
└── mr_CHARITE/
    ├── imagesTr/
    └── labelsTr/
```

Important (from GitHub): Ground-truth labels are required to generate prompt points during training. For inference without ground truth, "generate a fake ground-truth with the target region for prompt annotated." (i.e., a rough bounding mask indicating *where* to place automatic prompt points — it is not used for loss computation and does not need to be accurate.)

0.2.10 Turbo vs. Standard Comparison

Metric	Standard (base .pth)	Turbo (sam_med3d_turbo.pth)
Pre-training data	131 K masks	131 K + 44 additional datasets
Average Dice (1 prompt)	~0.75	~0.82+
Inference time	4–8 s	0.5–1.5 s
VRAM usage	~10 GB (FP32)	~4 GB (FP16)

0.3 Clinical Domain A: Coronary CCTA Segmentation (DISCHARGE)

0.3.1 DISCHARGE Trial Overview

Field	Value
Full name	Diagnostic Imaging Strategies for Patients with Stable Chest Pain and Intermediate Risk of Coronary Artery Disease
Reference	Dewey et al., NEJM 2022[4]
Design	Multicentre randomised controlled trial
Patients	3,561
Images	~25 M DICOM slices
Modality	Coronary Computed Tomography Angiography (CCTA)
Institution	Charité, Berlin (lead site)

0.3.2 SCOT-HEART Trial Overview

Field	Value
Full name	Scottish COmputed Tomography of the HEART Trial
References	Williams et al., Lancet 2015[5]; 10-year follow-up[6]
Design	Multicentre randomised controlled trial

Field	Value
Patients	4,146
Key result	CCTA-guided management → 41 % reduction in CHD death/MI (HR 0.59, 95 % CI 0.41–0.84) at 5-year follow-up[7]; 10-year follow-up data pending verification[6]
Modality	CCTA
Role in this project	Secondary validation dataset; extends generalisability beyond Charité site

0.3.3 Standardised Nomenclature (CAD-RADS 2.0 / AHA 17-segment compatible)[8]

Clinical Feature	Segmentation Class Label	Description	HU Range (contrast-enhanced CT)
Myocardium (LV)	`LV_MYO`	Muscular wall of left ventricle	50–120 HU
Endocardial Lumen	`Endocardial_Lumen`	Inner chamber blood pool	250–400 HU
Coronary Lumen	`Lumen_{LAD|RCA|LCx|...}`	Vessel blood pool — per AHA branch	300–400 HU
Outer Wall (EEM)	`VesselWall_EEM`	External elastic membrane boundary	— (morphological)
Calcified Plaque	`Plaque_Calc`	High-density stable plaque	> 130 HU (often 500–1000)
Fibrous Plaque	`Plaque_Fibrous`	Intermediate-density plaque	60–130 HU
Low-Attenuation Plaque	`Plaque_LAP`	Lipid-rich / necrotic core — **high risk**	< 60 HU

0.3.4 Why Coronary Arteries are the Hardest Segmentation Target

Challenge	Detail	Impact
Motion artefacts	Heart beats 60–80 bpm; RCA most affected	Blurred vessel edges despite ECG gating

Challenge	Detail	Impact
Small calibre	1.5–4 mm diameter	Partial volume effects at 0.5 mm resolution
Low-contrast plaque	Lipid-rich plaque 20–50 HU vs. lumen 300–400 HU	Nearly invisible on standard windowing
Blooming	Calcification > 130 HU causes beam hardening	Obscures adjacent soft plaque
Complex topology	Bifurcations, overlapping branches, tortuosity	Single-click prompts often insufficient
Dual-wall requirement	Must segment lumen AND outer wall separately	Wall thickness 0.5–3 mm = 1–5 voxels

0.3.5 SAM-Med3D-turbo Prompting Strategy for Coronary CCTA

0.3.5.1 A. Multi-Point Centerline Prompt ("String" Prompt) Standard single-click prompts fail for thin, tortuous vessels spanning 100+ slices. Instead:

1. User clicks **proximal ostium** (positive point)
2. User clicks **distal vessel tip** (positive point)
3. Model "fills in" the lumen tube between points
4. If mask leaks into **great cardiac vein** → place **negative point** on vein

```
# Coronary lumen segmentation with multi-point prompt
lumen_mask = model.segment(
    volume=cta_volume,
    points=[ostium_xyz, mid_vessel_xyz, distal_xyz],
    labels=[1, 1, 1],  # all positive
)

# If leakage detected → add negative point
lumen_mask_refined = model.segment(
    volume=cta_volume,
    points=[ostium_xyz, mid_vessel_xyz, distal_xyz, vein_xyz],
    labels=[1, 1, 1, 0],  # last point is negative
)
```

0.3.5.2 B. Dual-Wall Nested Segmentation (Lumen → EEM) Critical for calculating **stenosis %** and **plaque burden**:

> **Stenosis % definition:** Report **diameter stenosis** = (1 − D_min / D_ref) × 100, consistent with CAD-RADS 2.0 grading. D_ref is the mean diameter of the nearest healthy proximal and distal reference segments. Do **not** use area stenosis (more sensitive but not the CAD-RADS standard), and document the reference segment selection method explicitly — QAngio CT (MEDIS) uses interpolated reference; our pipeline must match this for Dice/correlation comparisons against expert measurements.

1. **Step 1:** Segment lumen (bright contrast, easier target)
2. **Step 2:** Use lumen mask as **dense prompt** → expand outward to EEM
3. **Result:** `vessel_wall = outer_mask & ~lumen_mask` → plaque volume

Implementation decision: SAM-Med3D does **not** natively support a `dense_prompt` argument in its published codebase. Two paths were evaluated: (a) custom prompt-encoder modification to accept a dense prior mask; (b) **surface-sampling** — erode the lumen mask, extract boundary voxels, and pass them as additional positive point prompts alongside the original ostium/distal pair. **Path (b) is the chosen approach for Phase 2**: no architecture change, immediately implementable, and validated in the SAM literature as effective for boundary-guided prompting. Concretely: sample ~30 points from the lumen mask surface using **farthest-point sampling** on the eroded boundary voxel set (maximises spatial coverage; avoids clustering all points on the nearest boundary face), append to `points` list with `label=1`. Path (a) (full dense conditioning) is deferred to Phase 4 if surface-sampling EEM Dice falls below 0.80 on the DISCHARGE validation split. The pseudocode below reflects the *intended* interface; replace `dense_prompt=lumen_mask` with the surface-sampled point list for the Phase 2 implementation.

Note for non-technical readers: the code below is illustrative pseudocode showing the intended design — it is not yet runnable production code.

```
# PSEUDOCODE — dense_prompt is not yet implemented in SAM-Med3D

# Step 1: Lumen
lumen_mask = model.segment(volume, points=[ostium, distal], labels=[1, 1])

# Step 2: Outer wall using lumen as prior (requires custom prompt encoder)
outer_mask = model.segment(
    volume,
    points=[ostium],
    dense_prompt=lumen_mask,  # NOT natively supported — must be implemented
    labels=[1]
)

# Step 3: Derive vessel wall
vessel_wall = outer_mask & ~lumen_mask
```

```
LAP_THRESHOLD = 0.04  # Research placeholder — no published consensus; tune against
HU_LAP_MAX    = 60    # < 60 HU = low-attenuation (lipid-rich / necrotic) plaque
HU_CALC_MIN   = 130   # > 130 HU = calcified plaque (Motoyama 2009; Maurovich-Horva

def characterise_plaque(volume, vessel_wall_mask):
    """Classify plaque components by HU value within the vessel wall mask."""
    hu_values = volume[vessel_wall_mask > 0]
```

```
    calcified    = (hu_values > 130).sum()
    fibrous      = ((hu_values >= 60) & (hu_values <= 130)).sum()
    lipid_rich   = (hu_values < 60).sum()
    total        = vessel_wall_mask.sum()

    return {
        "calcified_pct":  calcified / total * 100,
        "fibrous_pct":    fibrous / total * 100,
        "lipid_rich_pct": lipid_rich / total * 100,
        # lap_flag: LAP presence flag — threshold is a research placeholder.
        # No published consensus cutoff exists for LAP % of wall volume.
        # Motoyama 2009 uses qualitative LAP presence; Maurovich-Horvat 2014
        # uses absolute LAP volume. Tune against DISCHARGE outcomes data.
        "lap_flag": (lipid_rich / total) > LAP_THRESHOLD,
    }
```

0.3.5.3 C. Post-Processing: Plaque Characterisation by HU Thresholds

Plaque threshold provenance: The thresholds above (LAP < 60 HU, fibrous 60–130 HU, calcified > 130 HU) follow the scheme used in Motoyama et al. and Maurovich-Horvat et al.[9]. Some publications use stricter necrotic-core thresholds (< 30 HU) or LAP ≤ 50 HU — these are not universally standardised. Always cite the specific scheme used when reporting plaque composition in publications.

Positive remodeling: HU thresholds alone do not capture all high-risk plaque features. Positive remodeling (remodeling index > 1.1 per Motoyama 2009) is an independent high-risk feature that requires comparing lumen cross-sectional area to the EEM area at a reference segment. It is not derivable from HU values — it requires the dual-wall segmentation pipeline (lumen + EEM) to compute. This is an additional output the platform will enable once EEM segmentation is validated.

HU calibration warning: The thresholds above (< 60 LAP, 60–130 fibrous, > 130 calcified) are defined for **standard 120 kVp with soft-kernel reconstruction**. DISCHARGE is a multi-centre trial with varying tube voltages (80–140 kVp) and reconstruction kernels (soft vs. sharp). Sharp-kernel reconstruction shifts apparent HU upward and increases blooming around calcification. These thresholds require per-centre calibration against phantom measurements or paired soft/sharp kernel scans before cross-site plaque comparisons are valid. Record the reconstruction kernel and tube voltage for every DISCHARGE case in the metadata.

0.3.6 DISCHARGE-Specific Processing Considerations

Consideration	Detail
Scanner heterogeneity	Multi-centre trial → varying scanner vendors, protocols, contrast timing
Reconstruction kernels	Soft vs. sharp kernels affect HU accuracy for plaque
Contrast timing	Early arterial phase optimal; late phase reduces lumen-wall contrast
ECG gating	Prospective vs. retrospective gating affects motion artefact severity
Data format	DICOM (clinical) → convert to NIfTI.gz for model input
Annotations	MEDIS QAngio CT (expert contours) available for subset → ground truth

---

0.4 Clinical Domain B: Prostate mpMRI Segmentation

0.4.1 Imaging Standard

Multiparametric MRI (mpMRI) is the clinical standard for prostate imaging, using: - **T2-weighted (T2W):** Anatomical detail — zonal anatomy - **Diffusion-weighted imaging (DWI) + ADC map:** Cellularity — lesion detection - **Dynamic contrast-enhanced (DCE):** Vascularity — supplementary

Reporting follows **PI-RADS v2.1** (Prostate Imaging-Reporting and Data System)[10].

0.4.2 Anatomical Segmentation — The "Zones"

The prostate is divided into distinct zones with different MRI appearances and cancer risk:

Zone	Abbreviation	Cancer Risk	T2W Appearance	Clinical Role
Peripheral Zone	PZ	70–75 % of cancers	Bright (high signal)	Primary cancer surveillance region
Transition Zone	TZ	20–25 % of cancers	Heterogeneous (BPH nodules)	BPH assessment, cancer in older men
Central Zone	CZ	< 5 % of cancers	Low signal (dense stroma)	Indistinguishable from TZ on MRI — grouped with TZ per PI-RADS v2.1

Zone	Abbreviation	Cancer Risk	T2W Appearance	Clinical Role
Anterior Fibromuscular Stroma	AFMS	Non-glandular	Very low signal	Can be invaded by anterior tumours

0.4.3 Pathology Segmentation — The "Lesions"

When segmenting pathology, the target is **clinically significant prostate cancer (csPCa)**:

Lesion Type	Description	Clinical Significance
Index Lesion	Largest / most aggressive tumour	Primary target for biopsy and treatment
Satellite Lesions	Secondary foci (prostate cancer is often multifocal)	May affect treatment strategy
Extracapsular Extension (ECE)	Tumour breaches the prostatic capsule	Staging: T3a — affects surgical planning
Seminal Vesicle Invasion (SVI)	Tumour extends into seminal vesicles	Staging: T3b — impacts prognosis

0.4.4 Organs at Risk (OARs) for Radiotherapy

Structure	Abbreviation	Why Segment?
Neurovascular Bundles	NVB	Nerve-sparing surgery — preserve potency
Rectal Wall	Rectum_Wall	Monitor tumour–rectum distance
Bladder Neck	Bladder_Neck	Preserve urinary continence

0.4.5 Segmentation Class Labels for the AI Model

Segment Name	Class Label	Modality	Clinical Goal
Whole Gland	`Prostate_WG`	T2W	Volume / PSA density calculation
Peripheral Zone	`PZ`	T2W	Cancer surveillance background
Transition Zone	`TZ`	T2W	BPH assessment background
Suspicious Lesion	`Lesion_PIRADS_{3|4|5}`	DWI/ADC	Targeted biopsy (MR-US fusion)

Segment Name	Class Label	Modality	Clinical Goal
Seminal Vesicles	`SV`	T2W	Local staging (T3b)
Neurovascular Bundles	`NVB`	T2W	Nerve-sparing planning
Rectal Wall	`OAR_Rectum`	T2W	Radiotherapy constraints

0.4.6 SAM-Med3D-turbo Prompting Strategy for Prostate mpMRI

Context-dependent prompting: The same model must switch behaviour based on *where* the user clicks and *which sequence* is active.

0.4.6.1 Scenario 1: Anatomical Zone Segmentation (T2W)

```
1. User clicks bright outer rim on T2W axial
   → Model returns PZ mask
   → Expected Dice: ~0.90 (large, high-contrast structure)

2. User clicks central heterogeneous region on T2W
   → Model returns TZ mask

3. If mask leaks into rectum → place negative point on rectum
```

0.4.6.2 Scenario 2: Lesion Segmentation (DWI/ADC)

> **PI-RADS v2.1 dominant sequence rule:** DWI is the dominant determining sequence for peripheral zone (PZ) lesions; T2W is dominant for transition zone (TZ) lesions. The prompting strategy below routes all lesion clicks to the ADC map, which is appropriate for PZ. For TZ lesions, the click should be on T2W. The frontend should guide the radiologist accordingly based on anatomical zone.
>
> **Implementation note:** Step 2 below references using a prior PZ mask as a dense prompt. This follows the same surface-sampling decision made for Dual-Wall Nested Segmentation — sample ~30 points from the PZ mask boundary and pass as additional positive prompts. Full dense conditioning (path a) deferred to Phase 4.

```
1. User clicks hypointense spot on ADC map
   → Model returns lesion mask (PI-RADS ≥4 region)

2. [PLANNED] Use prior PZ mask as dense prompt context
   → Constrains lesion to within prostate boundary
  → Requires dense_prompt implementation (see [Dual-Wall Nested Segmentation](#corona
dual-wall-segmentation))

3. Measure lesion volume → maps to PI-RADS size criterion
```

> **Critical note:** SAM-Med3D was pre-trained on **MR data** (SA-Med3D-140K includes MR modalities). However, the dataset card does not specify which MR sequences or

whether prostate mpMRI is represented. Zero-shot performance on prostate zones may be moderate; fine-tuning on Charité prostate data will likely be necessary. Whole-gland segmentation (a large, well-defined structure) should work well zero-shot; zonal segmentation (PZ vs. TZ) is harder due to subtle signal differences.

0.4.7 SAM-Med3D-specific Challenges for Prostate mpMRI

Challenge	Detail	Mitigation
Multi-sequence input	SAM-Med3D accepts a single 3D volume; mpMRI has T2W + ADC + DCE	Run separate inferences per sequence; fuse masks post-hoc. Do not naively concatenate sequences — model was not trained on multi-channel input.
PZ–TZ boundary	Gradual signal transition, not a sharp edge	Use morphological priors (PZ wraps inferolaterally around TZ); negative prompts at suspected boundary to sharpen
Lesion detection vs. segmentation	PI-RADS lesions can be < 5 mm — smaller than SAM-Med3D's 128^3 patch at typical prostate resolution	Ensure patch is centred on clicked lesion; use higher-resolution input if available
Unknown pre-training coverage	SA-Med3D-140K dataset card does not confirm prostate mpMRI is represented	Treat prostate as low-confidence zero-shot; plan early fine-tuning on Charité cohort
Intensity normalisation (MRI)	MRI intensities are not calibrated across scanners (no HU equivalent)	Apply per-volume z-score normalisation independently for each sequence before inference
OAR segmentation	Neurovascular bundles (NVB) are extremely thin, especially at 1.5T; high-resolution T2W at 3T is the standard for NVB visualisation	Multi-point prompts along bundle; expect low zero-shot DSC — fine-tuning on 3T data required

0.4.8 Prostate vs. Coronary: Difficulty Comparison

Factor	Prostate	Coronary
Structure size	20–80 mL (large)	1.5–4 mm diameter (tiny)
Motion	None (static pelvis)	Cardiac motion (60–80 bpm)
Contrast	Good (gland vs. fat)	Variable (plaque vs. lumen)

Factor	Prostate	Coronary
Modality	MRI (multi-sequence)	CT (single phase)
Topology	Compact, roughly ellipsoidal	Thin, tortuous, branching tubes
Expected Dice	**> 0.90 (whole gland)**	**0.75–0.85 (lumen)**

0.5 Technical Challenges & Model-Aware Solutions

0.5.1 Challenge 1: Coronary Artery Motion Artefacts

Problem: Residual cardiac motion blurs vessel edges despite ECG gating. RCA most affected.

Solution:

```
def motion_robust_segmentation(volume_4d, heart_rate, prompt_points, prompt_labels)
    if heart_rate > 70:
        # Multi-phase reconstruction
        phases = extract_cardiac_phases(volume_4d, num_phases=10)
        masks = [model.segment(phase, points=prompt_points, labels=prompt_labels) f
        # Temporal median filter removes motion ghosts
        return np.median(masks, axis=0) > 0.5
    else:
        return model.segment(volume_4d[:,:,:,0], points=prompt_points, labels=promp
```

> **Topology warning:** Pixel-wise median of binary masks across 10 phases is **not** topologically safe for thin structures. A distal coronary voxel present in only 4/10 phases will be excluded by the median, potentially severing the centreline. Preferred alternative: select the optimal cardiac phase per SCCT guidelines[11] — typically mid-diastole (70–80 % R-R) at HR < 65 bpm, or end-systole (35–45 % R-R) at HR > 75 bpm — rather than fusing all phases. If multi-phase fusion is required, apply connected-component analysis post-median and re-link broken segments by dilation along the centreline.

Additional strategies: - Edge-preserving denoising (bilateral filter) pre-processing - Multi-point prompts every 5–10 mm along vessel to guide through corrupted regions - Auto-flag cases with edge sharpness < threshold for expert review

0.5.2 Challenge 2: Low-Attenuation Plaque Detection

Problem: Lipid-rich plaque (20–50 HU) nearly invisible against myocardium (50–70 HU).

Solution: Multi-stage approach: 1. Segment lumen and outer wall first (high-contrast boundaries) 2. Apply HU thresholding *within* the vessel wall mask 3. Use SAM-Med3D's ViT features for texture-based refinement (fine-tuning required)

0.5.3 Challenge 3: Dual-Wall Segmentation

Problem: Must segment both lumen and outer wall; wall thickness only 0.5–3 mm (1–5 voxels).

Solution: Sequential prompting (see Dual-Wall Nested Segmentation).

0.5.4 Challenge 4: Prostate Zone Boundaries

Problem: PZ-TZ boundary is a gradual signal transition, not a sharp edge.

Solution: - Train multi-class model on annotated Charité prostate data - Use morphological priors (PZ wraps around TZ inferolaterally) - Negative prompts at zone transitions to sharpen boundaries

0.5.5 Challenge 5: Model Limitations — Honest Assessment

Limitation	Impact	Mitigation
No coronary CTA in pre-training	Zero-shot may underperform	Fine-tune on DISCHARGE annotations
Binary mask output only	Cannot directly predict PI-RADS score	Post-processing pipeline (volume, ADC stats)
No `dense_prompt` in codebase	Lumen-as-prior strategy needs engineering	Custom prompt encoder modification
128^3 patch size constraint	Coronary arteries span > 128 voxels (300–400 voxels at 0.5 mm)	Sliding window with 50 % overlap; Gaussian-weighted blending at boundaries to prevent double-thick seams; connected-component check post-stitch
No multi-class output	One mask per inference call	Multiple sequential inferences per case
TTA not described	Test-time augmentation (axis flips, small rotations) commonly improves robustness on small structures	Evaluate TTA in Phase 3 as a robustness strategy; expect +2–5 % DSC on coronary lumen at the cost of 3–8× inference time

0.5.6 Challenge 6: Coronary Inference — Selective Patch Placement

Problem: Coronary arteries span 300–400 voxels at 0.5 mm isotropic resolution. A naive 50 % overlap sliding window over a 512^3 CCTA volume requires approximately 320 patches, yielding 160–480 s GPU time — 80–240× the 2 s target.

Solution: Centerline-guided selective patch placement. Only generate patches that contain centerline voxels; skip background patches entirely. Expected patch count: 20–40 (95 % reduction).

```
import numpy as np
from scipy.ndimage import binary_dilation

def get_coronary_patches(
```

```
    volume: np.ndarray,
    centerline_voxels: np.ndarray,  # (N, 3) array of voxel indices
    patch_size: int = 128,
) -> list[tuple[slice, slice, slice]]:
    """
    Return a minimal set of 128³ patch slices covering the entire centerline.
    Adjacent overlapping patches are merged to avoid redundant inference.
    """
    D, H, W = volume.shape
    half = patch_size // 2
    # _overlap_ratio(a, b): returns IoU of two slice-tuples as a float in [0,1]
    # _merge_slices(a, b, shape, patch_size): expands a to cover b, clamped to shap
    # _gaussian_kernel(n): returns an (n,n,n) array with unit-integral 3D Gaussian
    # (implementations omitted for brevity — standard numpy/scipy utilities)
    patches = []

    for point in centerline_voxels:
        z, y, x = point.astype(int)
        # Clamp from the far end so patches are always exactly patch_size³.
        # SAM-Med3D's 3D ViT uses fixed positional embeddings for 128³ input
        # and cannot accept variable-size patches without padding.
        z0 = max(0, min(D - patch_size, z - half)); z1 = z0 + patch_size
        y0 = max(0, min(H - patch_size, y - half)); y1 = y0 + patch_size
        x0 = max(0, min(W - patch_size, x - half)); x1 = x0 + patch_size
        sl = (slice(z0, z1), slice(y0, y1), slice(x0, x1))
        # Merge with last patch if overlap > 50 %
        if patches and _overlap_ratio(patches[-1], sl) > 0.5:
            patches[-1] = _merge_slices(patches[-1], sl, (D, H, W), patch_size)
        else:
            patches.append(sl)

    return patches


def run_selective_inference(volume, model, centerline_voxels, device):
    """
    Segment with Gaussian-weighted patch blending to prevent double-thick seams.
    """
    mask_accum = np.zeros(volume.shape, dtype=np.float32)
    weight_accum = np.zeros(volume.shape, dtype=np.float32)
    gaussian_weight = _gaussian_kernel(128)   # precomputed 128³ Gaussian

    patches = get_coronary_patches(volume, centerline_voxels)
    for sl in patches:
```

```
        patch = volume[sl]
        with torch.no_grad():
            # segment_patch: pseudocode for auto-prompted inference on a single
            # 128³ patch. In the actual MedIM API, replace with:
            #   model.segment(patch, points=[centerline_point_in_patch], labels=[1]
            # where the centerline point is re-expressed in patch-local coordinates
            pred = model.segment_patch(patch)   # returns (128,128,128) float
        mask_accum[sl] += pred * gaussian_weight[:pred.shape[0],
                                                 :pred.shape[1],
                                                 :pred.shape[2]]
        weight_accum[sl] += gaussian_weight[:pred.shape[0],
                                            :pred.shape[1],
                                            :pred.shape[2]]

    mask = (mask_accum / np.maximum(weight_accum, 1e-6)) > 0.5
    return mask
```

Performance expectation: 20–40 patches × 0.5–1.5 s each (FP16) = 10–60 s. Still above the 2 s interactive target. For interactive use, limit inference to the single patch centred on the user's click point and return a partial mask immediately; run full centerline-guided inference as a background refinement task via the Celery queue.

Prerequisite: A centerline must be available before inference. For interactive use, the user's two click points (ostium + distal) define a coarse 2-point centerline; for batch use, register against the coronary atlas template to seed the centerline automatically (see Workflow 2).

0.6 Clinical Workflows

0.6.1 Workflow 1: Interactive Coronary Segmentation (Radiologist)

1. Radiologist opens browser → logs in (Charité SSO)
2. Loads DISCHARGE CCTA scan from PACS
3. Clicks proximal LAD + distal tip → AI segments entire lumen in < 2 s
4. Optionally: places negative point to prune vein leakage
5. Clicks "Expand to Wall" → second inference → outer wall mask **[PLANNED — requires `dense_prompt` implementation; see Dual-Wall Nested Segmentation]**
6. Right panel shows: stenosis % (diameter stenosis, CAD-RADS 2.0), plaque composition, volume
7. Exports segmentation (NIfTI / DICOM-SEG / CSV report)

0.6.2 Workflow 2: Batch Processing for DISCHARGE Research

1. Research coordinator uploads 100 DISCHARGE cases

2. **Automated ostium seeding:** template-based registration to a reference coronary atlas provides initial prompt coordinates; cases where registration confidence is low are queued for manual prompt review before inference
3. AI processes overnight (Celery + multi-GPU batch mode)
4. Quality control: auto-flag cases with **disconnected mask components, mask volume outside 3σ of cohort distribution, or model confidence score below threshold** (ground truth is not available in batch mode — Dice cannot be computed)
5. Expert reviews flagged cases → corrections exported as corrected per-structure binary NIfTI masks → feed active-learning loop
6. Export refined segmentations for MACE (Major Adverse Cardiovascular Events: cardiac death, nonfatal MI, or unplanned revascularisation) prediction analysis

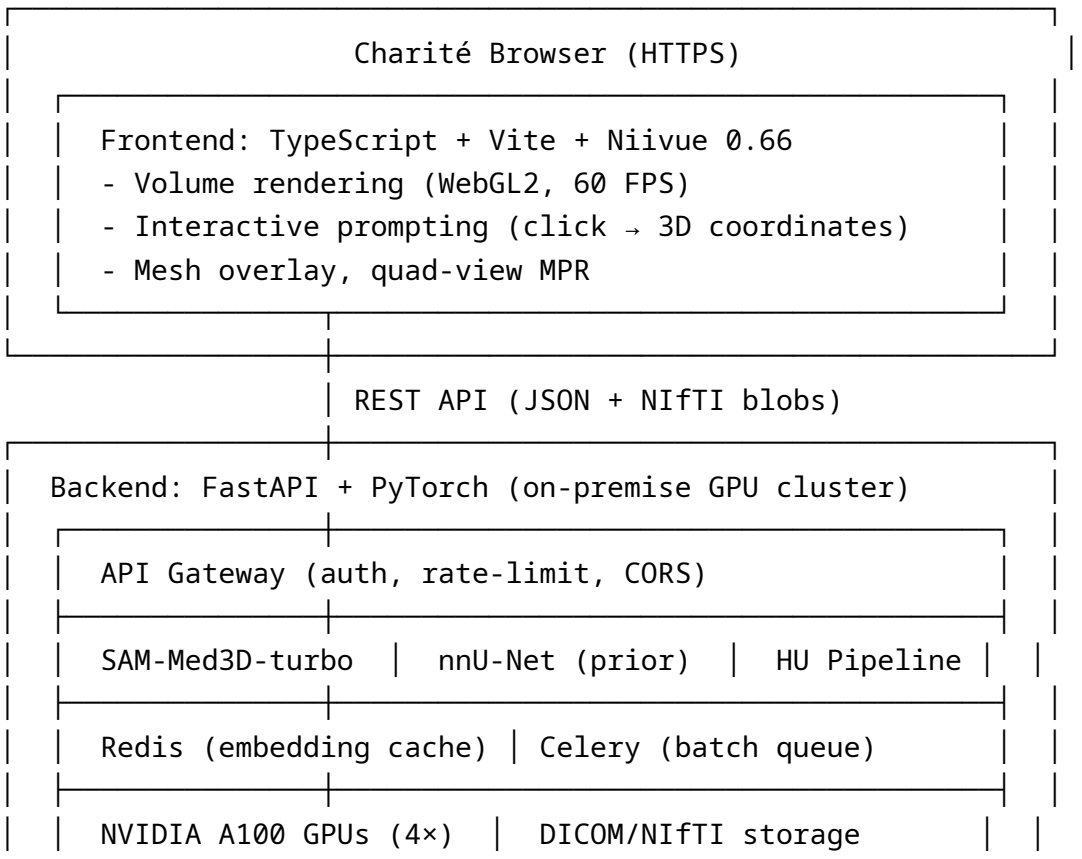
0.6.3 Workflow 3: MEDIS TXT + Mesh + Straightened MPR (Reference Contours)

1. Load CCTA volume (NIfTI.gz) in browser
2. Load MEDIS TXT file (expert contour rings)
3. Client-side mesh generation: parse TXT → NVMesh (50–100 ms)
4. Overlay lumen + vessel wall meshes on CCTA
5. Generate straightened MPR for longitudinal plaque assessment

---

0.7 System Architecture

0.7.1 High-Level Overview



```
│   └──────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────┘
```

0.7.2 Frontend Directory Structure

```
flow-segment-frontend/
├── src/
│   ├── main.ts                    # Entry point
│   ├── App.ts                     # Main app component
│   ├── components/
│   │   ├── NiivueViewer.ts        # Niivue canvas wrapper (single/quad)
│   │   ├── Toolbar.ts             # Top toolbar
│   │   ├── SegmentPanel.ts        # AI segmentation controls
│   │   ├── ResultsPanel.ts        # Plaque analysis / zone selector
│   │   ├── PromptHistory.ts       # Point prompt log + undo
│   │   └── MPRView.ts             # Multi-planar reconstruction
│   ├── services/
│   │   ├── api.ts                 # Backend API client
│   │   ├── niivue.ts              # Niivue init & config
│   │   ├── loader.ts              # NIfTI.gz + DICOM loading
│   │   ├── medisParser.ts         # MEDIS TXT parsing
│   │   ├── medisMeshDirect.ts     # Client-side contour→mesh
│   │   ├── straightenedMPR.ts     # CPR and transport-frame math
│   │   └── auth.ts                # LDAP/SSO
│   ├── types/
│   │   ├── volume.ts
│   │   ├── segmentation.ts
│   │   ├── mesh.ts
│   │   └── api.ts
│   └── utils/
│       ├── coordinates.ts         # 3D coordinate transforms
│       ├── meshGenerator.ts       # Marching cubes (vtk.js)
│       └── export.ts              # NIfTI / DICOM-SEG / CSV export
├── package.json
├── tsconfig.json
├── vite.config.ts
└── index.html
```

0.7.3 Backend Directory Structure

```
flow-segment-backend/
├── app/
│   ├── main.py                    # FastAPI application
│   ├── config.py                  # Environment config
│   ├── models/
```

```
│   │   ├── sam_med3d.py          # SAM-Med3D-turbo wrapper
│   │   ├── nnu_net.py            # nnU-Net (anatomical prior)
│   │   └── plaque_analyser.py    # HU-based plaque classification
│   ├── api/
│   │   ├── segment.py            # Segmentation endpoints
│   │   ├── volumes.py            # Volume management
│   │   ├── mesh.py               # Mesh generation (nii2mesh)
│   │   └── auth.py               # LDAP/SSO
│   └── services/
│       ├── cache.py              # Redis embedding cache
│       ├── dicom_processor.py    # DICOM → NIfTI (SimpleITK)
│       ├── mesh_generator.py     # nii2mesh wrapper
│       └── registration.py       # Elastix (longitudinal)
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
└── README.md
```

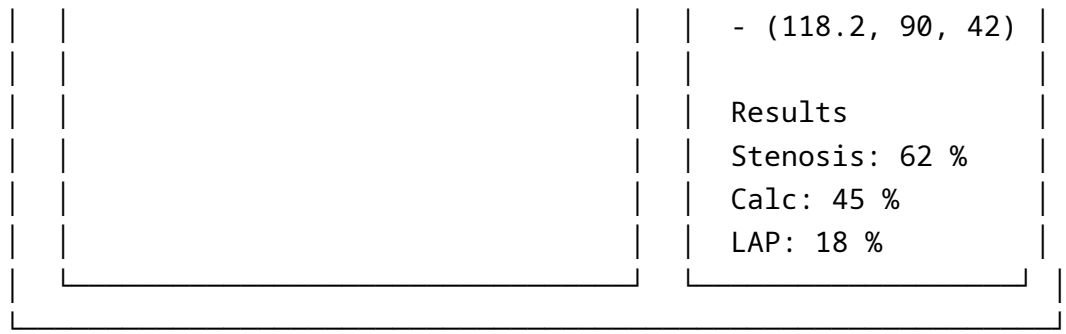
0.8 Frontend: Niivue Viewer & UI/UX[12]

0.8.1 Key Niivue v0.66.0 Capabilities

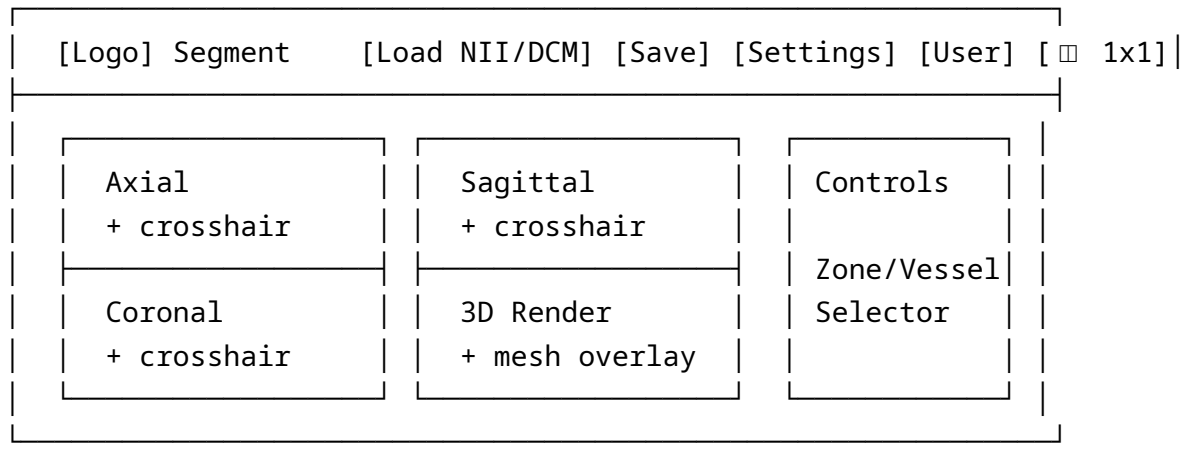
- WebGL2 rendering: 60 FPS for 512³ volumes
- `onLocationChange` / `onMouseUp` → capture 3D mm coordinates for prompts
- Multi-planar reconstruction (axial / coronal / sagittal sync)
- `nv.addMesh()` for 3D surface overlay with adjustable opacity
- Full TypeScript definitions
- In-browser DICOM via dcm2niix-wasm

0.8.2 UI Layout: Single View (Default)

```
┌──────────────────────────────────────────────────────────┐
│  [Logo] Segment    [Load NII/DCM] [Save] [Settings] [User] [◧ Quad]│
├──────────────────────────────────────────────────────────┤
│  ┌──────────────────────────────┐  ┌──────────────────┐  │
│  │                              │  │ Segmentation     │  │
│  │   Niivue 3D Canvas           │  │ ┌──────────────┐ │  │
│  │   [Click to place prompt]    │  │ │ ● Point      │ │  │
│  │                              │  │ │ □ Box        │ │  │
│  │                              │  │ │ ► Segment    │ │  │
│  │                              │  │ └──────────────┘ │  │
│  │                              │  │                  │  │
│  │                              │  │ Prompt History   │  │
│  │                              │  │ + (120.5, 85, 42) │  │
```



0.8.3 UI Layout: Quad View (MPR + 3D)



0.8.4 Design Principles

- **Dark theme** (#1a1a1a) — reduces eye strain for long sessions
- **Minimal chrome** — maximise canvas area
- **Radiologist-first** — optimised for clinical workflow
- Primary: Blue (#3b82f6), Accent: Green (#10b981), Warning: Orange (#f59e0b), Error: Red (#ef4444)

0.8.5 Keyboard Shortcuts

Key	Action
`P`	Place positive prompt point
`N`	Place negative prompt point
`Enter`	Run segmentation with current prompts
`Z`	Undo last prompt point
`Escape`	Cancel current prompt session
`Q`	Toggle single / quad view
`W` / `L`	Adjust window / level (hold + drag)
`Space`	Toggle mesh overlay visibility

0.8.6 General-Purpose Rotatable Volume Viewer

Port of legacy `viewer.py` (PyQtGraph) to WebGL:

- Free rotation via mouse drag (Rodrigues' rotation formula)
- Three-panel orthogonal views (YZ, XZ, XY) — all rotate together
- Drag modes: Rotation (0), Paint/Label (1), Window/Level (2)
- Real-time volume resampling (SimpleITK → WebGL equivalent)
- Target: 30–60 FPS during rotation

0.9 Backend: Inference Pipeline & API

> **Clinical readers:** this section and the following MEDIS Parser and Patch Placement sections contain implementation-level Python/TypeScript code. They are intended for software engineers and AI researchers. Clinical readers may skip directly to Deployment, Performance & Security.

0.9.1 API Endpoints

```
# Segmentation (AI)
POST /api/segment/point        # Point-based prompting (SAM-Med3D)
POST /api/segment/box          # Bounding box prompting
POST /api/segment/refine       # Refine with negative points

# Volume Management
POST /api/volumes/upload       # Upload DICOM / NIfTI
GET  /api/volumes/{id}         # Retrieve processed volume

# Mesh Generation
POST /api/mesh/from-mask       # Mask → MZ3 mesh (nii2mesh)
POST /api/mesh/quick           # Fast preview (marching cubes)

# MEDIS TXT Processing
POST /api/medis/upload         # Upload MEDIS TXT
POST /api/medis/to-mesh        # Convert contours to mesh

# Straightened MPR
POST /api/straighten/create    # Centerline → straightened volume

# Batch Processing
POST /api/batch/process        # Queue batch of cases
GET  /api/batch/{job_id}       # Job status

# Authentication
POST /api/auth/login           # LDAP / Charité SSO
```

```
GET  /api/health                # Health check + GPU status
```

0.9.2 HU Intensity Normalisation (CCTA Pre-processing)

SAM-Med3D was trained on data with modality-specific intensity normalisation. Raw CCTA HU values span −1000 to +3000, but only the cardiovascular window is relevant. Feeding unnormalised values will degrade segmentation quality.

Required pre-processing before inference:

```python
def normalise_ccta(volume: np.ndarray, clip_min: float = -100, clip_max: float = 70
    """
    Clip to cardiovascular window and normalise to [0, 1].
    clip_min=-100 HU: excludes air; clip_max=700 HU: captures calcification peak.
    Adjust clip_max to 400 HU for lumen-only work (excludes dense calcium blooming)
    """
    volume = np.clip(volume, clip_min, clip_max)
    volume = (volume - clip_min) / (clip_max - clip_min)  # → [0, 1]
    return volume.astype(np.float32)
```

> **Note:** The exact normalisation used in SAM-Med3D's SA-Med3D-140K training pipeline is not fully documented for CT modalities. The values above are a clinically reasonable starting point. Before any fine-tuning, run the sweep below to select the optimal config. For prostate mpMRI, normalise each sequence (T2W, ADC) independently using per-volume z-score normalisation.

Normalisation sweep protocol (B1 — blocking before fine-tuning): Run all six configs on 10 held-out DISCHARGE cases with available MEDIS ground truth; select the config with highest mean coronary lumen DSC.

Config	`clip_min`	`clip_max`	Rationale
A	−100	700	Cardiovascular window (default above)
B	−200	1000	Wider; preserves calcification peak
C	−100	400	Lumen-only; excludes dense calcium blooming
D	−300	1500	Matches common SA-Med3D-140K CT pre-processing guesses
E	z-score (μ=0, σ=1)	per-volume	Modality-agnostic baseline

Config	clip_min	clip_max	Rationale
F	nnU-Net v2 scheme	Foreground-voxel (HU > −500) percentile clip [0.5th–99.5th] + z-score over same foreground voxels	De facto standard for CT deep learning (2024–2025)[1]

```
SWEEP_CONFIGS = [
    {"clip_min": -100, "clip_max":  700},   # A: cardiovascular window
    {"clip_min": -200, "clip_max": 1000},   # B: wider, preserves calcium peak
    {"clip_min": -100, "clip_max":  400},   # C: lumen-only
    {"clip_min": -300, "clip_max": 1500},   # D: broad SA-Med3D-140K guess
    {"mode": "zscore"},                      # E: z-score over foreground voxels
    {"mode": "nnunet_v2"},                   # F: nnU-Net v2 CT scheme
]

def normalise_ccta_sweep(volume: np.ndarray, cfg: dict) -> np.ndarray:
    mode = cfg.get("mode")
    if mode == "zscore":
        # Compute stats over foreground voxels only (exclude air, HU < -500)
        fg = volume[volume > -500]
        return ((volume - fg.mean()) / (fg.std() + 1e-8)).astype(np.float32)
    if mode == "nnunet_v2":
        # nnU-Net v2 CT normalisation: percentile clip over foreground voxels,
        # then z-score over those same foreground voxels.
        fg = volume[volume > -500]
        lo, hi = np.percentile(fg, 0.5), np.percentile(fg, 99.5)
        v = np.clip(volume, lo, hi)
        return ((v - fg.mean()) / (fg.std() + 1e-8)).astype(np.float32)
    lo, hi = cfg["clip_min"], cfg["clip_max"]
    v = np.clip(volume, lo, hi)
    return ((v - lo) / (hi - lo)).astype(np.float32)
```

Record the winning config in `config.py` as `NORM_CLIP_MIN` / `NORM_CLIP_MAX` before Phase 3 zero-shot evaluation. Do not change it after the validation split is locked.

0.9.3 Segmentation Endpoint (Core)

```
import re
import threading
from typing import Optional
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
import nibabel as nib
```

```
import numpy as np
import torch
import medim

router = APIRouter()

# Allowlist pattern: alphanumeric, hyphens, underscores only — no path traversal
_VOLUME_ID_RE = re.compile(r'^[A-Za-z0-9_\-]{1,128}$')

class SegmentRequest(BaseModel):
    volume_id: str
    coordinates: list[list[float]]   # [[x,y,z], ...] in mm — one or more points
    labels: Optional[list[int]] = None  # 1=positive, 0=negative per point; default

# Model loaded once at startup (inside FastAPI lifespan event in production).
# _model_lock serialises access across FastAPI's sync thread pool workers —
# PyTorch inference is not thread-safe with a shared model instance.
model = medim.create_model(
    "SAM-Med3D", pretrained=True,
    checkpoint_path="app/models/checkpoints/sam_med3d_turbo.pth"
)
device = "cuda" if torch.cuda.is_available() else "cpu"
model = model.to(device)
if device == "cuda":
    model = model.half()
model.eval()
_model_lock = threading.Lock()
# NOTE: the lock serialises all inference to one request at a time.
# For production multi-user workloads (5-10 concurrent radiologists),
# replace with a dedicated inference server (Triton Inference Server or
# TorchServe) or multiple model replicas behind a load balancer.

# Use def (not async def): FastAPI runs sync endpoints in a thread pool,
# keeping the event loop free during blocking GPU inference.
@router.post("/api/segment/point")
def segment_point(req: SegmentRequest):
    # Path traversal guard — reject any volume_id that is not a safe token
    if not _VOLUME_ID_RE.match(req.volume_id):
        raise HTTPException(status_code=400, detail="Invalid volume_id format.")
    if not req.coordinates:
        raise HTTPException(status_code=400, detail="At least one coordinate is req
    if any(len(pt) != 3 for pt in req.coordinates):
        raise HTTPException(status_code=400, detail="Each coordinate must be [x, y,
    if req.labels is not None and len(req.labels) != len(req.coordinates):
```

```
        raise HTTPException(status_code=400, detail="labels length must match coord

    # Load volume from cache/disk.
    # VOLUMES_BASE_PATH ("/data/volumes") is a deployment-specific config value
    # set in config.py — not hard-coded in production.
    vol_nii = nib.load(f"/data/volumes/{req.volume_id}.nii.gz")
    volume = vol_nii.get_fdata(dtype=np.float32)

    # Normalise HU values to cardiovascular window before inference.
    # NOTE: normalise_ccta() assumes CT (HU values). For MRI volumes (prostate),
    # use per-volume z-score normalisation instead — this endpoint will need a
    # modality parameter once prostate mpMRI support is added in Phase 5.
    volume = normalise_ccta(volume)

    # Convert mm → voxel coordinates for each point
    inv_affine = np.linalg.inv(vol_nii.affine)
    voxel_points = [(inv_affine @ [*pt, 1])[:3] for pt in req.coordinates]

    effective_labels = req.labels if req.labels is not None else [1] * len(voxel_po

    # Serialise inference: _model_lock prevents concurrent GPU calls from
    # multiple thread-pool workers corrupting model state.
    with _model_lock:
        with torch.no_grad():
            mask = model.segment(volume, points=voxel_points, labels=effective_labe

    # Return mask as NIfTI
    mask_nii = nib.Nifti1Image(mask.astype(np.uint8), vol_nii.affine)
    # ... serialize and return
```

0.10 MEDIS TXT Reference Pipeline

0.10.1 Format Description

MEDIS TXT contains vessel wall contours from MEDIS QAngio CT: - **Lumen** contours: define inner wall (blood pool boundary) - **VesselWall** contours: define outer wall (including plaque) - Each contour: group label, slice distance, N points in 3D mm coordinates

0.10.2 Parser (TypeScript)

```
export interface MedisContour {
  group: "Lumen" | "VesselWall";
  sliceDistance: number;           // mm along vessel
```

```
  points: { x: number; y: number; z: number }[];
}

export function parseMedisTxt(content: string): MedisContour[] {
  const lines = content.split("\n");
  const contours: MedisContour[] = [];
  let current: Partial<MedisContour> = {};
  let points: { x: number; y: number; z: number }[] = [];

  for (const line of lines) {
    if (line.startsWith("# group:")) {
      current.group = line.split(":")[1].trim() as "Lumen" | "VesselWall";
    } else if (line.startsWith("# SliceDistance:")) {
      current.sliceDistance = parseFloat(line.split(":")[1]);
    } else if (line.startsWith("# Contour index:")) {
      if (current.group && points.length > 0) {
        contours.push({ ...current, points } as MedisContour);
      }
      // Reset both points AND current so the next contour does not inherit
      // stale group/sliceDistance values from the previous block.
      current = {};
      points = [];
    } else if (line.trim() && !line.startsWith("#")) {
      const [x, y, z] = line.trim().split(/\s+/).map(Number);
      if (!isNaN(x)) points.push({ x, y, z });
    }
  }
  if (current.group && points.length > 0) {
    contours.push({ ...current, points } as MedisContour);
  }
  return contours;
}
```

0.10.3 Direct Client-Side Mesh Construction (< 100 ms)

Contour rings are connected into a tube mesh directly in the browser — no backend round-trip needed. Algorithm: connect ring N to ring N+1 with triangle pairs.

Performance comparison: | Method | Latency | Network | |———|———|———| | Backend (build-stl.py → STL → download) | 500–2000 ms | Required | | **Client-side (TXT → NVMesh)** | **50–100 ms** | **None** |

0.11 Mesh Generation Strategies

0.11.1 Three Approaches

Approach	Method	Speed	Quality	Use Case
Ultra-Simple	Connect contour rings → STL	< 50 ms	Faceted	MEDIS export
Client-side vtk.js	Marching cubes on mask	< 1 s	Good	Interactive preview
Server nii2mesh → MZ3	Decimation + smoothing	1–3 s	High	Final visualisation

0.11.2 Recommended Web Format: MZ3

- 3–5× smaller than PLY, 10× smaller than STL
- Binary, gzip-compressed, native Niivue support
- Target: < 5 MB per mesh, 50K–200K triangles, 60 FPS rendering

---

0.12 Centerline Extraction & Straightened MPR (CPR)

0.12.1 Overview

Straightened MPR (Curved Planar Reformation) "unfolds" a tortuous vessel into a straight view. Essential for assessing stenosis and plaque distribution along the entire vessel length.

Three steps: 1. **Centerline extraction** — centroid of lumen contours (from MEDIS) or Voronoi skeletonisation / VMTK vessel-tree extraction (from AI mask; VMTK handles bifurcations and provides radius estimates at each centreline point) 2. **Bishop (parallel transport) frame** — compute Tangent (T) then propagate Normal (N) and Binormal (B) without relying on curvature (see Mathematical Foundation for why Frenet-Serret N must not be used) 3. **Cross-section sampling** — extract perpendicular slices → stack into straightened volume

0.12.2 Mathematical Foundation

Tangent (finite differences):

```
T[i] = normalize(centerline[i+1] - centerline[i-1])  // central difference
```

Normal — Bishop (parallel transport) frame:

> **Why not Frenet-Serret:** The Frenet-Serret principal normal `N = normalize(dT/ds)` is undefined and flips 180° in low-curvature (near-straight) vessel segments, producing rotating or jumping cross-sections. Bishop frame (rotation-minimizing frame) propagation[13,14] avoids this by carrying an initial normal forward without relying on curvature.

```
// Initialise with any vector perpendicular to T[0]
N[0] = arbitrary_perpendicular(T[0])

// Propagate: project previous N onto the plane perpendicular to new T
for i in 1..n-1:
    projected = N[i-1] - dot(N[i-1], T[i]) * T[i]
    // Guard: if T[i] ≈ N[i-1] (degenerate), projected → 0 → divide-by-zero.
    // Fall back to re-seeding from an arbitrary perpendicular.
    if |projected| < epsilon:
        N[i] = arbitrary_perpendicular(T[i])
    else:
        N[i] = normalize(projected)
```

Binormal: `B[i] = T[i] × N[i]`

Cross-section point: `Q(u,v) = P[i] + u·N[i] + v·B[i]`

0.12.3 Interactive Controls

- **Position slider:** Select centerline point (0 → N-1)
- **Rotation slider:** Rotate cross-section around T (Rodrigues' formula)
- **Zoom slider:** Adjust cross-section FOV
- **Quad-view:** CCTA overview | cross-section | straightened MPR | 3D mesh

---

0.13 Deployment, Performance & Security

0.13.1 Hardware Requirements

0.13.1.1 Development Environment

Component	Minimum	Recommended	Current Setup
GPU	RTX 3060 (8 GB)	RTX 2080 Ti (11 GB)	2× RTX 2080 Ti (11 GB each)
CPU	6-core	8+ core	Modern workstation
RAM	16 GB	32 GB+	Sufficient
Storage	500 GB SSD	1 TB+ NVMe	Adequate
CUDA	11.8+	12.2+	CUDA 12.2

Development use cases: model testing, single-user interactive segmentation, DISCHARGE subset processing, active-learning experiments.

0.13.1.2 Production Environment (Charité Clinical Deployment)

Component	Minimum	Recommended	Notes
GPU Cluster	2× RTX 4090 (24 GB)	4× A100/H100 (40–80 GB)	Concurrent clinical users
CPU	16-core	32+ core	FastAPI + preprocessing
RAM	64 GB	128 GB+	Multiple 3D volumes in memory
Storage	10 TB+	50 TB+	DISCHARGE + clinical data
Network	10 Gbps	25 Gbps+	Fast volume transfers
Redundancy	RAID 10	Distributed storage	Clinical data safety

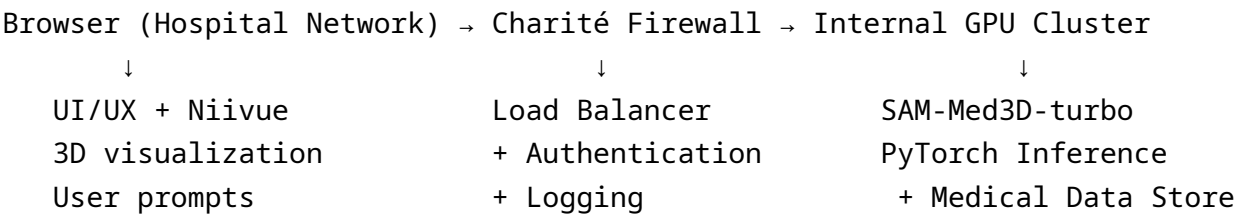
Production requirements: GDPR compliance (all data on-premise), 99.9 % uptime, 5–10 concurrent radiologists, full DISCHARGE batch processing, automated backup and failover.

0.13.2 GDPR & Deployment Constraints

WARNING: IN-HOUSE BACKEND MANDATORY FOR CLINICAL USE

For any clinical deployment, the SAM-Med3D-turbo model **MUST** run on Charité's on-premise infrastructure:

Requirement	Reason	Alternative
On-premise GPU servers	GDPR — patient data cannot leave hospital	Not allowed: cloud inference
Charité network	Secure medical data transmission	Not allowed: public internet
Hospital authentication	User access control and audit trails	Not allowed: anonymous access
Medical device certification	Clinical safety and regulatory compliance	Not allowed: research-only deployment



Non-clinical use cases (anonymised data allowed): research demos with cloud GPU; local development with sample datasets; open-source contributions via GitHub with synthetic data; educational browser-only UI with mock backend.

0.13.3 Infrastructure

Component	Technology
Containerisation	Docker + NVIDIA Container Toolkit
GPU	4× NVIDIA A100 (80 GB each)
Embedding cache	Redis (sub-second for repeated prompts)
Batch queue	Celery + Redis broker (each worker pinned to one GPU via `CUDA_VISIBLE_DEVICES`; 4 workers = 4 GPUs = 4 concurrent cases)
Authentication	LDAP / Charité SSO
Compliance	GDPR (all data on-premise, no cloud)

0.13.4 Performance Targets

Metric	Target
Single-click → mask	< 2 s end-to-end
Embedding computation (first prompt)	~1 s
Subsequent prompts (cached embedding)	< 0.5 s
Mesh generation (MZ3)	< 3 s
Batch throughput	~50 cases / hour (4 GPUs) — bottleneck is DICOM→NIfTI preprocessing, not GPU
CTA volume memory	~268 MB ($512^3 \times 2$ bytes = 268,435,456 bytes)

0.13.5 Memory Budget

Component	Size
CTA volume (512^3, int16)	~268 MB
SAM-Med3D-turbo (FP16)	~4 GB VRAM
Embedding cache (per volume)	~50–500 MB (estimate — depends on patch count and feature map size; to be measured). Assumes image-encoder output is cached in Redis between prompts, so only the prompt encoder + mask decoder run on subsequent clicks. Verify this is supported by the MedIM API before relying on sub-second repeat-prompt performance.
Straightened volume ($64^2 \times 200$)	~8 MB
Mesh (MZ3, per vessel)	< 5 MB

0.14 Research Roadmap & Milestones

0.14.1 Phase 1: MVP — MEDIS TXT Visualisation (Weeks 1–4)

- ☐ MEDIS TXT parser + client-side mesh in Niivue
- ☐ Centerline extraction + straightened MPR with **Bishop (parallel transport) frame** (not Frenet-Serret — see Mathematical Foundation)
- ☐ Quad-view layout with interactive sliders
- ☐ NIfTI.gz / DICOM loading

Phase 1 → Phase 2 gate (must all pass before starting Phase 2): - MEDIS TXT parser handles all DISCHARGE vessel files without crash - Bishop-frame CPR renders in Niivue without rotation artefacts at straight segments - Quad-view interactive sliders run at ≥ 30 FPS on reference workstation - Zero failing TypeScript strict-mode errors in frontend build

0.14.2 Phase 2: SAM-Med3D Integration (Weeks 5–8)

- ☐ Backend: load turbo checkpoint, expose `/api/segment/point`
- ☐ Frontend: click → prompt → mask overlay
- ☐ Redis embedding cache for sub-second repeated prompts
- ☐ **Implement surface-sampling dense prompt** — erode lumen mask, sample ~30 boundary voxels uniformly, pass as additional positive points; see Challenge 6 for implementation detail
- ☐ Dual-wall sequential segmentation pipeline (depends on above)
- ☐ Selective centerline-guided patch inference for coronary volumes (see Challenge 6)

Phase 2 → Phase 3 gate (must all pass before starting Phase 3): - `/api/segment/point` returns a mask in ≤ 5 s (single patch, interactive path) on dev GPU - Surface-sampling dual-wall pipeline produces non-empty EEM mask on ≥ 5/5 test cases - Selective patch inference runs on a 512³ CCTA volume in ≤ 90 s end-to-end (dev GPU)

0.14.3 Phase 3: DISCHARGE Evaluation (Weeks 9–12)

- ☐ **Zero-shot baseline — myocardium first:** `LV_MYO` is the closest match to ACDC pre-training targets; establish DSC baseline here before coronary lumen (see Verified Performance). Compare against TotalSegmentator[15] as an additional zero-shot reference for LV myocardium.
- ☐ Fix held-out validation split: 100 DISCHARGE cases, stratified by scanner site, locked before any fine-tuning begins
- ☐ Zero-shot baseline on coronary lumen (held-out DISCHARGE cases)
- ☐ Quantify DSC, HD95, diameter stenosis % correlation vs. MEDIS QAngio CT expert measurements — report against thresholds in Evaluation Metrics
- ☐ Identify failure modes (motion, calcification, bifurcations, patch-boundary seams)

Phase 3 → Phase 4 gate (must all pass): - LV myocardium zero-shot DSC ≥ 0.82 on held-out split - Coronary lumen zero-shot DSC ≥ 0.70 on held-out split - Normalisation sweep (B1) complete; optimal config selected and documented

0.14.4 Phase 4: Fine-tuning & Active Learning (Weeks 13–20)

- ☐ Fine-tune SAM-Med3D on DISCHARGE annotations (nnU-Net-style data prep)
- ☐ Active-learning loop[16]: expert corrections → re-training triggered when correction buffer reaches N = 50 new cases (event-driven), or weekly — whichever comes first. Weekly cadence is a reasonable starting point; the 2024–2025 trend favours event-triggered fine-tuning to avoid unnecessary retraining at low correction volumes. **Correction data model:** a "correction" is a full per-structure binary NIfTI mask saved by the radiologist after local brush edits in the browser. The backend must convert the in-browser voxel edits (sparse difference from AI mask) into a complete corrected NIfTI matching Data Format for Fine-tuning before ingestion into the fine-tuning pipeline.
- ☐ **Catastrophic forgetting mitigation:** weekly fine-tuning on new corrections will degrade performance on earlier cases if the full training set is not replayed. Implement a replay buffer (random sample of 10–20 % of previous fine-tuning cases included in every weekly batch) or elastic weight consolidation before the loop is deployed.
- ☐ Benchmark against task-specific nnU-Net v2 baseline

0.14.5 Phase 5: Prostate Extension (Weeks 21–28)

- ☐ Adapt pipeline for prostate mpMRI (multi-sequence input); add modality parameter to `/api/segment/point` for MRI z-score normalisation
- ☐ Zone-specific class labels (PZ / TZ / lesion)
- ☐ Establish nnU-Net v2 task-specific baseline on Charité prostate cohort (analogous to the coronary nnU-Net v2 baseline) before evaluating SAM-Med3D-turbo
- ☐ Validate on Charité prostate cohort; report DSC/HD95 against thresholds in Evaluation Metrics

0.14.6 Phase 6: Clinical Validation & Publication (Weeks 29–36)

- ☐ Prospective reader study (Dice vs. time vs. inter-observer)
- ☐ Open-source release + MedSegFM competition baseline
- ☐ Publication: "Foundation-model-assisted coronary CCTA segmentation at scale"

0.14.7 Quarterly Research Milestones

Quarter	Milestone	Status
Q1 2026 (ends 2026-03-31)	Zero-shot baseline on DISCHARGE + MVP deployed	In progress - update before end of quarter
Q2 2026	Active-learning loop running; Dice ≥ 0.80 on coronary lumen	Pending
Q3 2026	Prospective reader study; prostate pipeline validated	Pending
Q4 2026	Open-source release; conference/journal submission	Pending

0.15 Related Medical Segmentation Models

Model	Year	Dimension	Modalities	Prompts	Notes
SAM (Meta)	2023	2D	Natural images	Point/box/text	No medical training, 2D only
SAM-Med2D	2023	2D	Medical (slice-wise)	Point/box	2D; cannot capture volumetric context
MedSAM	2023	2D	Medical (slice-wise)	Box only	Simpler architecture, box prompts only
SAM-Med3D-turbo	**2024**	**3D**	**Medical (volumetric)**	**3D point**	**Our choice** — native 3D, ECCV Oral, 91 M params
SAM 2 (Meta)	2024	2D+T / pseudo-3D	Natural images + video	Point/box/mask	Video memory tokens[17]; adapted for slice-by-slice 3D propagation in MedSAM-2 — strong alternative, lacks native isotropic 3D encoder

Model	Year	Dimension	Modalities	Prompts	Notes
MedSAM-2	2024–25	Pseudo-3D (slice stack)	Medical (volumetric)	Point/box	SAM 2 adapted for medical volumes[18]; competitive on organ segmentation; to be tracked for coronary evaluation
SegVol	2024	3D	CT (volumetric)	Text + point + box	Semantic text prompts[19]; trained on 90 K CT volumes; strong on CT organs; no published coronary results
TotalSegmentator	2023	3D	CT (automatic)	None	104-structure automatic CT segmentation[15]; relevant zero-shot baseline for LV myocardium; nnU-Net v1 based

Model	Year	Dimension	Modalities	Prompts	Notes
nnU-Net v2	2024	3D	Medical (task-specific)	None (automatic)	Gold-standard baseline[1]; not prompt-able; best for single-task fine-tuned performance

Why SAM-Med3D-turbo over SAM 2 / MedSAM-2: SAM 2 processes volumes as a sequence of 2D slices with memory propagation, not a native isotropic 3D encoder. For thin, tortuous coronary arteries where the vessel may subtend only 1–2 voxels per axial slice, 2D-first processing loses cross-plane spatial context that a 3D ViT encoder retains. MedSAM-2 should be tracked and included in Phase 3 benchmarking. SegVol's text prompts are not suitable for fine-grained coronary anatomy. nnU-Net v2 remains the task-specific performance ceiling and will be used as the benchmark baseline throughout.

This document is the living foundation of the Charité Segment Platform research project. All claims about SAM-Med3D are verified against the published paper (arXiv:2310.15161), official GitHub README, and HuggingFace model/dataset cards. Critical caveats about zero-shot performance on coronary CTA and prostate mpMRI (neither directly evaluated in the paper) are noted throughout. Full bibliography in `plan/references.bib`*.*

References

1. Isensee F, Wald T, Ulrich C, Baumgartner M, Roy S, Maier-Hein K, et al. nnU-net revisited: A call for rigorous validation in 3D medical image segmentation [Internet]. 2024. doi:10.48550/arXiv.2404.09556

2. Wang H, Guo S, Ye J, Deng Z, Cheng J, Li T, et al. SAM-Med3D: Towards general-purpose segmentation models for volumetric medical images. In: European conference on computer vision (ECCV) workshops: Bioimage computing [Internet]. Springer; 2024. p. 1–18. doi:10.1007/978-3-031-72469-6_1

3. Isensee F, Jaeger PF, Kohl SAA, Petersen J, Maier-Hein KH. nnU-net: A self-configuring method for deep learning-based biomedical image segmentation. Nature Methods. 2021;18(2):203–11. doi:10.1038/s41592-020-01008-z

4. DISCHARGE Trial Group. Computed tomography or invasive coronary angiography in patients with stable chest pain. New England Journal of Medicine. 2022;386(19):1795–806. doi:10.1056/NEJMoa2200963

5. SCOT-HEART Investigators. CT coronary angiography in patients with suspected angina due to coronary heart disease (SCOT-HEART): An open-label, parallel-group, multicentre trial. The Lancet. 2015;385(9985):2383–91. doi:10.1016/S0140-6736(15)60291-4

6. Williams MC, Hunter A, Shah ASV, Assi V, Lewis S, Mangion K, et al. Coronary CT angiography-guided management of patients with stable chest pain: 10-year results of the SCOT-HEART randomised controlled trial. The Lancet. 2025;406(10471):47–59. doi:10.1016/S0140-6736(24)02679-5

7. SCOT-HEART Investigators. Coronary CT angiography and 5-year risk of myocardial infarction. New England Journal of Medicine. 2018;379(10):924–33. doi:10.1056/NEJMoa1805971

8. Cury RC, Leipsic J, Abbara S, Achenbach S, Berman D, Bittencourt M, et al. CAD-RADS™ 2.0 — 2022 coronary artery disease–reporting and data system: An expert consensus document of the society of cardiovascular computed tomography (SCCT), the american college of cardiology (ACC), the american college of radiology (ACR), and the north america society of cardiovascular imaging (NASCI). Journal of Cardiovascular Computed Tomography. 2022;16(6):536–57. doi:10.1016/j.jcct.2022.07.001

9. Maurovich-Horvat P, Ferencik M, Voros S, Merkely B, Hoffmann U. Comprehensive plaque assessment by coronary CT angiography. Nature Reviews Cardiology. 2014;11(7):390–402. doi:10.1038/nrcardio.2014.60

10. Turkbey B, Rosenkrantz AB, Haider MA, Padhani AR, Villeirs G, Macura KJ, et al. Prostate imaging reporting and data system version 2.1: 2019 update of prostate imaging reporting and data system version 2. European Urology. 2019;76(3):340–51. doi:10.1016/j.eururo.2019.02.033

11. Leipsic J, Abbara S, Achenbach S, Cury R, Earls JP, Mancini GBJ, et al. SCCT guidelines for the interpretation and reporting of coronary CT angiography: A report of the society of cardiovascular computed tomography guidelines committee. Journal of Cardiovascular Computed Tomography. 2014;8(5):342–58. doi:10.1016/j.jcct.2014.07.003

12. Rorden C, Taylor H, Gallivan K, Hanayik T. Niivue: A WebGL2-based medical image viewer [Internet]. GitHub; 2024. doi:10.5281/zenodo.8320937

13. Wang W, Jüttler B, Zheng Y, Liu Y. Computation of rotation minimizing frames. ACM Transactions on Graphics. 2008;27(1):1–18. doi:10.1145/1330511.1330512

14. Kanitsar A, Ropinski T, Fleischmann D, König A, Gröller ME. CPR - curved planar reformation. In: IEEE visualization. IEEE; 2002. p. 413–20. doi:10.1109/VIS.2002.5930066

15. Wasserthal J, Breit HC, Meyer MT, Pradella M, Hinck D, Sauter AW, et al. TotalSegmentator: Robust segmentation of 104 anatomical structures in CT images. Radiology: Artificial Intelligence. 2023;5(5):e230024. doi:10.1148/ryai.230024

16. Budd S, Robinson EC, Kainz B. A survey on active learning and human-in-the-loop deep learning for medical image analysis. Medical Image Analysis. 2021;71:102062. doi:10.1016/j.media.2021.102062

17. Ravi N, Gabeur V, Hu YT, Hu R, Ryali C, Ma T, et al. SAM 2: Segment anything in images and videos. arXiv preprint arXiv:240800714. 2024; doi:10.48550/arXiv.2408.00714

18. Zhu Y, Zhang Y, Zhang T, Wang Z, Zhang R, Ding K, et al. MedSAM-2: Segment anything in 3D medical images and videos. arXiv preprint arXiv:240800874. 2024; doi:10.48550/arXiv.2408.00874

19. Du Y, Bai F, Huang T, Zhao B. SegVol: Universal and interactive volumetric medical image segmentation. arXiv preprint arXiv:231113385. 2024; doi:10.48550/arXiv.2311.13385